数字逻辑与处理器基础·第二次大作业·实验报告

无 48 吴泓锦 2024010654

〇、承诺书

第零部分　承诺书

在提交的实验报告首页，应附上已填写完成的承诺书截图。承诺书内容如下：

本人郑重承诺：已按照大作业文档中“提交要求”部分的说明，对所有相关文件进行整理，并按规定格式进行压缩打包提交。若因文件格式、命名或提交方式等不符合要求而导致丢分，本人愿承担全部责任。

同时，作业中如涉及使用人工智能（AI）生成或辅助完成的内容，均已在对应题目处明确标注所使用的 AI 提供商及模型名称，并在实验报告的附录中以截图形式提供与 AI 的完整对话记录，确保内容真实、来源清晰、过程可追溯。

承诺人：吴泓锦

日　期：2026.6.7

一、基础练习

1-1. 系统调用（AI 辅助：OpenAI-GPT-5.5 Thinking）

程序功能：

1. 申请一个 8 byte 整数的内存空间，从“a.in”读取两个整数，并将其写入“a.out”；
2. 从键盘输入一个整数 i，向屏幕打印 i+10 的计算结果。

测试结果：

（程序测试前已将“a.in”文件存放到“Mars4_5.jar”同目录下）

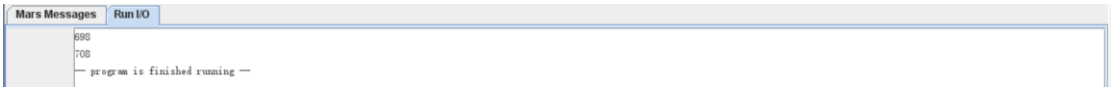
程序执行前，利用 VSCode 官方 Hex Editor 插件查看“a.in”所存数据：



这里，每 4 个字节构成一个整数，并且按照小端序存放，因此存放的两个数分别是：00000001，对应十进制 1；0000000A，对应十进制 10。这与说明文档是一致的。

程序测试后，可以看到显示运行成功，其中键盘输入 698，成功输出 708：





此时再查看“a.in”文件，未发生改变：



而同目录下新建了“a.out”文件，打开得到：

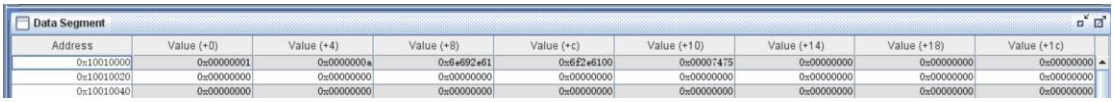


可以发现已成功写入这两个整数。

此时，寄存器$t0 存储的值为计算后的 708。

$a3	7	0x00000000
$t0	8	0x000002c4
$t1	9	0x00000000

内存地址 0x10010000 处依次存放十进制整数 1、10、“a.in”和“a.out”两个字符串。



1-2. 循环，分支

程序功能：

1. 输入整数 i 和 j（j 不为 0）；
2. 对 i 取相反数，若 j 为负数，也对 j 取相反数；
3. 循环 j 次，每次循环中 i 自增 1，最后打印 i 的最终结果，数值上应当等于-i+|j|。

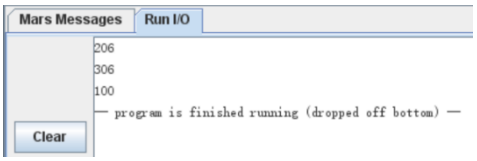
测试结果：

程序测试后，可以看到显示运行成功：



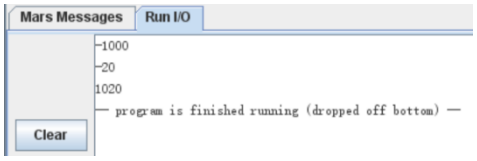
分别测试了三组数据点：

第一组，i=206，j=306，理论输出 100：



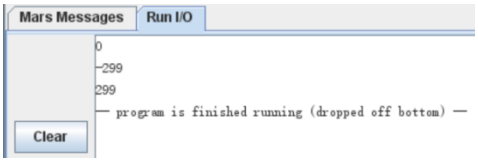
$at	1	0x00000000
$v0	2	0x00000064
$v1	3	0x00000000
$a0	4	0x00000064

第二组，i=-1000，j=-20，理论输出 1020：



$at	1	0x00000000
$v0	2	0x000003fc
$v1	3	0x00000000
$a0	4	0x000003fc

第三组，i=0，j=-299，理论输出 299：



$at	1	0x00000000
$v0	2	0x0000012b
$v1	3	0x00000000
$a0	4	0x0000012b

1-3. 数组，指针

程序功能：

1. 输入数组长度 n 和 n 个整数，创建整数数组 a；

2. 将 a 中元素逐个加 1，并逆序存储在 a 中，最后顺次打印数组 a 最终存储的数据。

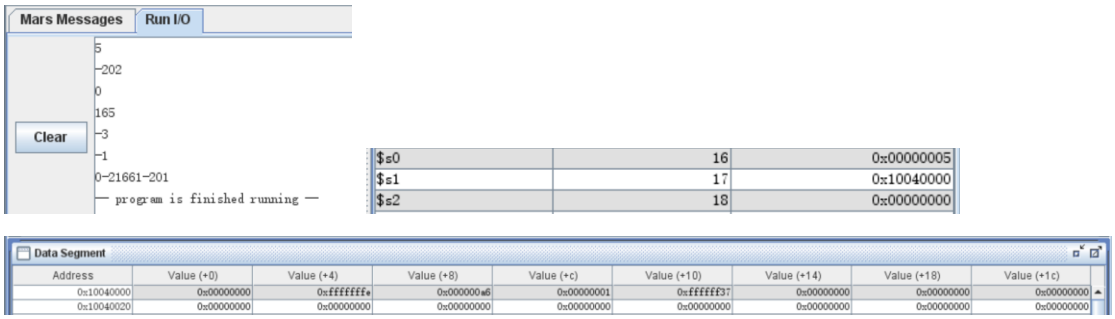
测试结果：

程序测试后，可以看到显示运行成功：

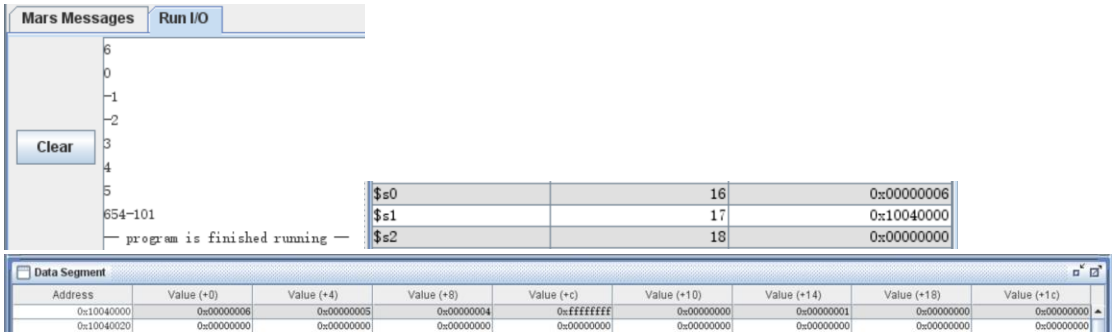


分别测试了两组数据点：

第一组，数组长度为 5（奇数），原数组中存储数据依次为：-202，0，165，-3，-1。最终数组应当顺次存放 0，-2，166，1，-201，而由于原 C++程序输出时各数据之间无空格，故理论输出为“0-21661-201”，与实际结果相符。$s1 寄存器存放数组首地址 0x10040000，可以看到内存相应部分存放数组最终数据：



第二组，数组长度为 6（偶数），原数组中存储数据依次为：0，-1，-2，3，4，5。最终数组应当顺次存放 6，5，4，-1，0，1。与第一组同理，理论输出为“654-101”，与实际结果相符。同理寄存器$s1 存放数组首地址，可以看到内存相应部分存放数组最终数据：



1-4. 函数调用

程序功能：

通过递归，计算并打印斐波那契数列的某一项的值。

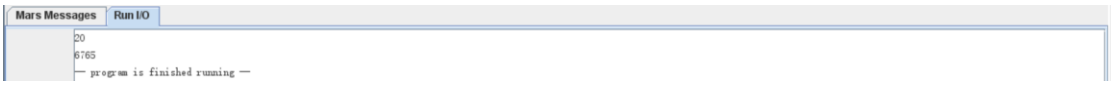
测试结果：

程序测试后，可以看到显示运行成功：

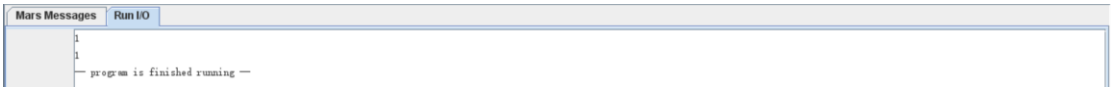


分别测试了三个数据点：

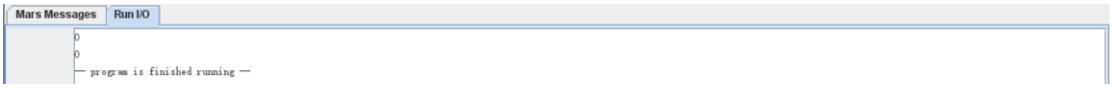
第一个，输入 n=20，理论输出 6765，实际输出正确：



第二个，输入 n=1，理论输出 1，实际输出正确：



第三个，输入 n=0，理论输出 0，实际输出正确：



二、实战应用

2-1. 严格递增子列划分（AI 辅助：OpenAI-GPT-5.5 Thinking）

寄存器和变量之间分配关系：

寄存器	变量	具体说明
$s0	N	序列长度
$s1		输入文件描述符
$s2	&a[1]	第二个输入数据的地址，即序列首个数地址
$s3	dp	二维数组 dp 的首地址
$s4	N+1	dp 每一行的长度
$s5		常量 INF，表示一个很大的数
$s6	i	外层循环变量
$s7	j	内层循环变量
$t0-$t2		临时变量，用于计算下标或偏移
$t3		临时变量，用于保存地址或结果
$t4	a[i-1]	当前外层循环中的 a[i-1]
$t5	a[i]	当前外层循环中的 a[i]
$t6		存放当前要访问的 dp 单元地址
$t7		当前 dp 单元原值
$t8		候选转移值，常为 dp[i-1][j-1]
$t9		条件判断结果
$a0-$a2		syscall 参数
$v0		syscall 编号或返回值

过程调用操作：

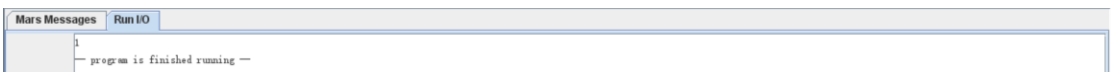
本程序没有进行自定义过程调用。程序主体均在 main 中顺序执行，通过循环和分支完成输入读取、DP 表初始化、状态转移和结果输出，未使用 jal 调用自定义函数，因此不涉及过程调用操作。

最终程序输出：

程序测试后，可以看到显示运行成功：



测试样例可以输出正确结果 1：



此外，还测试了“2 9 9”“4 1 1 1 1”“4 4 3 2 1”“4 1 1 2 2”“6 1 2 1 2 3 3”“6 1 2 1 2 1 2”“6 1 2 3 4 5 6”“8 -2 100 -1 101 0 102 1 103”等数据点。

2-2. 树型网络放大器（AI 辅助：OpenAI-GPT-5.5 Thinking）

寄存器和变量之间分配关系：

寄存器	main		dfs	
	变量	**具体说明**	**变量**	**具体说明**
$s0	n	网络结点数量	u	当前结点
$s1	文件描述符		parent	父结点
$s2	edge_cnt	边编号	edge_to_parent	当前结点到父结点边长
$s3	输入缓冲区指针		sons	当前结点儿子数量
$s4			e	当前扫描的边编号
$s5			v	当前邻接点
$s6			len	当前边长
$s7			临时地址或临时结果	
$t0-$t9	临时变量，用于地址计算、循环计数、比较结果等			
$a0-$a2	syscall 参数		调用参数 u、parent、edge_to_parent	
$v0	syscall 编号、syscall 返回值、函数返回值			
$ra			过程返回地址	压栈保存
$sp			栈指针	递归保存现场

过程调用操作：

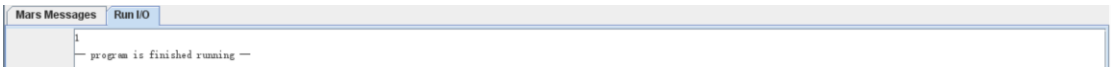
通过 jal 调用递归过程 dfs。调用时，通过$a0、$a1、$a2 分别传递当前结点、父结点和父边长度；进入过程后，先将$ra 和$s0-$s7 压栈保存，并把参数转存到$s0-$s2 中。在过程内部遍历当前结点所有邻接边，对每个非父结点的子结点再次使用 jal dfs 递归调用，利用子结点结果更新当前结点的 f 和 g。过程结束时恢复栈中保存的寄存器和返回地址，释放栈空间，并通过 jr $ra 返回。

最终程序输出：

程序测试后，可以看到显示运行成功：



测试样例可以输出正确结果 1：



此外，还测试了单结点、星型树、长链多次放大等情形（具体数据由 AI 给出）。

附录、与 AI 对话截图

AI 提供商与模型名：OpenAI-GPT-5.5（Thinking 模式）

下面用 ChatGPT Exporter 导出聊天记录：

MIPS汇编程序解析

06/24/2026 00:26:21

[Image: image(111).png] 我之前已经学过一些MIPS的基本指令，现在需要完整撰写汇编程序，自学一些知识，请根据图片解释一下什么是数据段？什么是正文段？这里每一句指令分别是什么意...

这张图展示的是一个完整 MIPS 汇编程序的基本结构：先定义数据，再写程序指令。MIPS 汇编通常分成两大部分： .data # 数据段：放变量、数组、字符串等 .text # 正文段：放真正执行的指令 1....

[Image: image(112).png] 图中这个程序中，为什么要把$ra的返回地址先存到$s7中，最后再重新赋回$ra?

这里先把 $ra 存到 $s7，最后再赋回 $ra，本质上是在保护返回地址。 1. $ra 是什么？ $ra 是 return address register，也就是返回地址寄存器。 当执行函数调用指令： jal some_function 时，...

下面是一段从文件读取数据并写入另一个文件的MIPS汇编代码，请先整体分析各个寄存器和段落标志的作用，再逐句或逐段分析代码的含义和作用： .data in_buff: .space 512 out_buff: .space 5...

下面这段程序的整体功能可以概括为： 从 example.in 文件中读取最多 512 字节到 in_buff；把 in_buff 中的一组 word 数据复制到 out_buff；复制过程中把每个整数打印到控制台，并打印逗号；...

问题1：MIPS汇编程序中".globl"是什么意思？是不是可加可不加？ 问题2：在利用MIPS程序打开文件时，如果其不存在，是否会直接自动创建？而在其写入文件时，是在先前文件中已有的数据后...

问题1：.globl 是什么意思？是不是可加可不加？ .globl 是 global 的缩写，意思是： 把某个标签声明为"全局符号"，使它可以被其他文件、链接器或模拟器入口机制识别。 常见写法是： .globl ma...

在我完成第一个实验时，老师给定的输入文件a.in应该存放在什么路径下？应当与我的exp1_1.asm文件在同一目录下，还是应当与Mars4_5.jar（即MARS启动入口）放在同一目录下?

一般来说，在 MARS 中使用： input_file: .asciiz "a.in" 这种相对路径文件名时，a.in 的查找位置通常不是由 exp1_1.asm 所在目录决定，而是由 MARS 程序运行时的工作目录决定。 对你现在最常...

如果我想查看a.in文件中存的具体是什么数，应当怎么打开？现在我用vscode打开，显示"此文件是二进制文件或使用了不受支持的文本编码，所以无法在文本编辑器中显示。"，强行打开后显示是...

这是正常现象：你的 a.in 很可能不是文本文件，而是二进制文件。 也就是说，它里面存的不是字符形式的： 17 1 2 3 4 5 ... 而是直接按 4 字节整数 word 存储的数据，例如： 0x00000011...

汇编大作业说明文档-2026春(1).pdf [Image: image(113).png] 现在我正在完成这份大作业，正在完成实验1，利用vscode及hex editor插件打开a.in文件后，显示结果如图，根据大作业要求，这应...

根据大作业说明文档，实验 1-1 的 a.in 是一个二进制格式输入文件，里面包含 1 和 10 两个整数。说明文档第 5 页明确写到：实验 1-1 要"从 a.in 读取两个整数"，并且测试样例 a.in "包含 1 和 10 ...

在实验1-3中，通过syscall的9号指令申请分配内存时，系统分配的内存地址是与自己的系统有关吗？为什么我的系统给我分配到的内存地址是0x0000000a，这应当不在正常的text或dynamic data...

不是。实验 1-3 中 syscall 9 分配出来的地址不应当与你自己的 Windows/Mac/Linux 系统真实内存地址有关，它是在 MARS 模拟器的虚拟 MIPS 内存空间中分配的地址。 正常情况下，syscall 9 返...

如果希望在vscode中解析查看dat二进制文件，之前所说的hex editor能不能用?

可以，VS Code 的 Hex Editor 可以用来查看 .dat 二进制文件。但要注意：Hex Editor 只能直接显示原始字节，不会自动帮你把它"解析成整数序列"。你需要按实验规定的数据格式自己分组理解。...

在MIPS中，系统调用syscall有哪些？请把所有编号的系统调用全部罗列出来

Thinking 🌐 搜索MARS MIPS系统调用服务表 hwlabnitc.github.iodpetersanderson.github.iogithub.comstudocu.vnasm-...

在实验2-1中，我希望给dp表设置一个非常大的初始值代表不可达状态，即INF，这个数设置多少比较合适?

在实验 2-1 里，INF 的作用通常是给 dp 表设置一个"足够大、表示当前状态不可达"的初值。因此它应该满足： INF > 输入序列中所有可能出现的实际数值 同时还要避免在 MIPS 有符号比较中被当...

对于实验2-2树型网络，请你帮我想几个全面的测试数据，尽可能考虑corner cases，给出十进制输入、理论输出和输入的十六进制形式（用于存进dat文件）

Thinking ● 确定问题和规则 我正在理清用户的问题。似乎是在问如何计算信号经过树形网络的传播。目标是找出所需的最小放大器数量，并考虑信号强度和衰减的规定。 ● 确定问题和规则 🌐 ...

MIPS汇编：语法

```
    --              .data 0x10010000  # 以下部分定义数据段（可省略地址）
0x10010000      n:  .word 5           # 0x10010000处存1个字，值为5。后面程序可用n来访问此地址
0x10010008      str: .asciiz "Hello!" # 0x10010008处存放以'\0'结尾字符串。str用来访问首地址
    --              .align 2          # 对齐到4字节。否则下一个地址为0x1001000f，会出错！
0x10010010      arr: .space 8         # 0x10010010处存8个字节（2个字）。arr用来访问首地址
    --              .text 0x00400000  # 以下部分定义正文段（可省略地址）
0x00400000          la $s0, n         # 伪指令la：用于加载相应label的地址到相应寄存器
0x00400004          la $s1, arr
0x00400008          lw $t0, 0($s0)    # 使用访存指令去读写数据段的数据
0x0040000c          sw $t0, 0($s1)
0x00400010          addi $t0, $t0, 1
0x00400014          sw $t0, 4($s1)
```

我之前已经学过一些MIPS的基本指令，现在需要完整撰写汇编程序，自学一些知识，请根据图片解释一下什么是数据段？什么是正文段？这里每一句指令分别是什么意思（lw、sw、addi这些指令我明白，但是小数点开头的我不知道是什么意思）？如何理解地址的迭代？

这张图展示的是一个**完整 MIPS 汇编程序的基本结构**：先定义数据，再写程序指令。MIPS 汇编通常分成两大部分：

```
.data
    # 数据段：放变量、数组、字符串等

.text
    # 正文段：放真正执行的指令
```

---

1. 什么是数据段 `.data`？

`.data` 表示：**下面开始是数据段**。

数据段用于存放程序运行时要用的数据，例如：

```
.data 0x10010000
```

意思是：从地址 `0x10010000` 开始存放数据。

在图中：

```
.data 0x10010000
```

表示后面定义的变量、字符串、数组等，会从内存地址 `0x10010000` 开始依次存放。

例如：

```
n: .word 5
```

表示定义一个变量 `n`，它占用一个 word，也就是 **4 字节**，值为 `5`。

因此：

```
0x10010000    n:    .word 5
```

表示：

> *地址 `0x10010000` 处存放了一个 32 位整数，值为 5。*
> *以后程序可以用标签 `n` 来访问这个地址。*

---

2. 什么是正文段 `.text`？

`.text` 表示：**下面开始是正文段，也就是程序指令段**。

正文段存放真正会被 CPU 执行的指令，例如：

```
lw
sw
addi
beq
j
```

在图中：

```
.text 0x00400000
```

表示程序指令从地址 `0x00400000` 开始存放。

所以后面的指令：

```
la $s0, n
la $s1, arr
lw $t0, 0($s0)
sw $t0, 0($s1)
addi $t0, $t0, 1
sw $t0, 4($s1)
```

会从 `0x00400000` 开始，每条指令占 4 字节，依次存放。

---

3. 小数点开头的语句是什么？

小数点开头的语句不是普通 CPU 指令，而是**汇编伪指令 / 汇编指示符**，英文常叫 assembler directive。

它们不会都直接变成机器指令，而是告诉汇编器：

> *这里要定义数据、切换段、分配空间、进行对齐等。*

图中出现了这些：

```
.data
.word
.asciiz
.align
.space
.text
```

逐个解释如下。

---

4. 每一句是什么意思？

① `.data 0x10010000`

```
.data 0x10010000
```

进入数据段，并指定数据段从地址 `0x10010000` 开始。

后面定义的数据都会从这个地址开始，按顺序向后存放。

---

② `n: .word 5`

```
n: .word 5
```

这里有两部分：

```
n:
```

是标签，表示这个位置的地址名字叫 `n`。

```
.word 5
```

表示在这里存放一个 word，也就是 4 字节整数，值为 5。

所以：

```
n: .word 5
```

整体意思是：

> *定义一个名为 `n` 的整数变量，初值为 5。*

因为它从 `0x10010000` 开始，所以：

```
n = 0x10010000
```

这个地址中存放的数据是：

```
5
```

---

③ `str: .asciiz "Hello!"`

```
str: .asciiz "Hello!"
```

`str:` 是标签，表示这个字符串的首地址叫 `str`。

`.asciiz` 表示定义一个以 `\0` 结尾的字符串。

```
"Hello!"
```

本身有 6 个字符：

```
H e l l o !
```

每个字符 1 字节，共 6 字节。

`.asciiz` 会自动在末尾加一个空字符 `\0`，所以总共占：

```
6 + 1 = 7 字节
```

因此：

```
str: .asciiz "Hello!"
```

意思是：

> *在内存中存放字符串 `"Hello!\0"`，标签 `str` 表示该字符串首地址。*

图中它从 `0x10010008` 开始。

---

④ `.align 2`

```
.align 2
```

这一句表示**地址对齐**。

在 MIPS 中，很多数据访问要求地址对齐。例如：

```
lw
sw
```

访问 word 时，地址最好是 4 的倍数。

`.align 2` 的意思是：

> *将下一个数据的起始地址对齐到 `2^2 = 4` 字节边界。*

也就是说，下一个数据的地址必须是 4 的倍数。

例如：

```
0x1001000c
0x10010010
0x10010014
```

这些都是 4 字节对齐的地址。

为什么图中说：

```
否则下一个地址为 0x1001000f，会出错
```

是因为前面的字符串长度不是 4 的整数倍，如果不对齐，后面定义的 word 数组可能会从非 4 字节对齐的地址开始。这样用 `lw` 或 `sw` 访问时可能出错。

---

⑤ `arr: .space 8`

```
arr: .space 8
```

`arr:` 是标签，表示这个数组空间的首地址叫 `arr`。

`.space 8` 表示：

> *从当前位置开始，预留 8 个字节的空间。*

8 字节等于 2 个 word，因为：

```
1 word = 4 字节
8 字节 = 2 word
```

所以：

```
arr: .space 8
```

可以理解为：

> *定义一个数组 `arr`，它有 8 字节空间，可以存放两个 32 位整数。*

图中由于经过 `.align 2` 对齐，所以 `arr` 从：

```
0x10010010
```

开始。

---

⑥ `.text 0x00400000`

```
.text 0x00400000
```

进入正文段，并指定程序指令从地址 `0x00400000` 开始存放。

从这里开始，后面的就是 CPU 实际执行的指令。

---

⑦ `la $s0, n`

```
la $s0, n
```

`la` 是 load address，加载地址。

它是伪指令，不是真正的 MIPS 基本指令。

意思是：

```
$s0 = n 的地址
```

因为：

```
n = 0x10010000
```

所以执行后：

```
$s0 = 0x10010000
```

注意，它不是把 `n` 中的值 5 放进 `$s0`，而是把 `n` 的地址放进 `$s0`。

---

⑧ `la $s1, arr`

```
la $s1, arr
```

意思是：

```
$s1 = arr 的首地址
```

因为：

```
arr = 0x10010010
```

所以执行后：

```
$s1 = 0x10010010
```

---

⑨ `lw $t0, 0($s0)`

```
lw $t0, 0($s0)
```

意思是：

```
$t0 = Memory[$s0 + 0]
```

由于：

```
$s0 = 0x10010000
```

所以：

```
$t0 = Memory[0x10010000]
```

而 `0x10010000` 处存放的是变量 `n` 的值，也就是 5。

因此执行后：

```
$t0 = 5
```

---

⑩ `sw $t0, 0($s1)`

```
sw $t0, 0($s1)
```

意思是：

```
Memory[$s1 + 0] = $t0
```

由于：

```
$s1 = 0x10010010
$t0 = 5
```

所以：

```
Memory[0x10010010] = 5
```

也就是说，把 5 存入 `arr` 的第一个 word。

可以理解为 C 语言中的：

```
arr[0] = 5;
```

⑪ `addi $t0, $t0, 1`

```
addi $t0, $t0, 1
```

意思是：

```
$t0 = $t0 + 1
```

前面 `$t0 = 5`，所以执行后：

```
$t0 = 6
```

⑫ `sw $t0, 4($s1)`

```
sw $t0, 4($s1)
```

意思是：

```
Memory[$s1 + 4] = $t0
```

由于：

```
$s1 = 0x10010010
$t0 = 6
```

所以：

```
Memory[0x10010014] = 6
```

也就是把 6 存入 `arr` 的第二个 word。

可以理解为 C 语言中的：

```
arr[1] = 6;
```

5. 程序整体做了什么?

这个程序整体相当于：

```c
int n = 5;
char str[] = "Hello!";
int arr[2];

arr[0] = n;
arr[1] = n + 1;
```

执行完之后，内存大致是：

地址	内容	说明
`0x10010000`	`5`	`n`
`0x10010008`	`"Hello!\0"`	字符串 `str`
`0x10010010`	`5`	`arr[0]`
`0x10010014`	`6`	`arr[1]`

6. 如何理解地址的迭代?

所谓"地址的迭代"，可以理解为：

> *汇编器在数据段或正文段中，每放入一项内容，当前地址就会向后移动相应的字节数。*

数据段中的地址迭代

从：

```
.data 0x10010000
```

开始。

第一句：

```
n: .word 5
```

`.word` 占 4 字节，所以如果从 `0x10010000` 开始，后面地址至少前进 4 字节：

```
0x10010000 \~ 0x10010003
```

接下来地址变为：

```
0x10010004
```

但是图中 `str` 放在 `0x10010008`，说明中间可能还有对齐或模拟器默认分配习惯。不过核心原则是：

```
每定义一个数据，地址就按它占用的字节数向后移动。
```

字符串：

```
.asciiz "Hello!"
```

占 7 字节，所以地址会后移 7。

然后：

```
.align 2
```

把下一个地址调整到 4 的倍数。

最后：

```
.space 8
```

预留 8 字节，所以地址再后移 8。

正文段中的地址迭代

从：

```
.text 0x00400000
```

开始。

MIPS 中每条普通指令固定 4 字节，因此地址依次是：

```
0x00400000
0x00400004
0x00400008
0x0040000c
0x00400010
0x00400014
```

也就是每条指令地址加 4。

图中：

```
0x00400000    la $s0, n
0x00400004    la $s1, arr
0x00400008    lw $t0, 0($s0)
0x0040000c    sw $t0, 0($s1)
0x00400010    addi $t0, $t0, 1
0x00400014    sw $t0, 4($s1)
```

就是这个规律。

不过要注意，`la` 是伪指令，在真实机器中可能会被翻译成多条真正的机器指令；图中是教学简化，把它当成一条指令展示。

---

7. 一个核心区别：地址和值不要混淆

这段程序中最容易混淆的是：

```
la $s0, n
```

和

```
lw $t0, 0($s0)
```

它们的区别是：

```
la $s0, n
```

得到的是地址：

```
$s0 = 0x10010000
```

而：

```
lw $t0, 0($s0)
```

得到的是该地址中存放的值：

```
$t0 = Memory[0x10010000] = 5
```

所以：

```
la
```

是“取地址”，类似 C 语言中的：

```
p = &n;
```

而：

```
lw
```

是“按地址取值”，类似：

```
x = *p;
```

这正是汇编程序中理解数据访问的关键。

```
        .data
arr1:   .word 1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17
arr2:   .space 68
        .text
main:   add $s7, $ra, $zero     # 保存返回地址到$s7
         la $s5, arr1            # $s5=&A
        la $s6, arr2            # $s6=&B
        addi $s0, $zero, 0x11   # Size(A)=Size(B)=0x11
loop:   addi $s0, $s0, -1       # 计数
        sll $s1, $s0, 2         # 换算地址
        add $s2, $s1, $s5       # 计算A[]偏移量,送到$s2
        lw $s3, 0($s2)          # 读出A[]中的值
        add $s2, $s1, $s6       # 计算B[]偏移量,送到$s2
        sw $s3, 0($s2)          # 写到B[]中去
        bnez $s0, loop
        add $ra, $zero, $s7     # 返回主调过程
        jr $ra
```

每个数4Bytes, 1 word

功能：将数组A的值依次拷贝到数组B中

图中这个程序中，为什么要把$ra的返回地址先存到$s7中，最后再重新赋回$ra?

这里先把 `$ra` 存到 `$s7`，最后再赋回 `$ra`，本质上是在**保护返回地址**。

---

1. `$ra` 是什么?

`$ra` 是 **return address register**，也就是返回地址寄存器。

当执行函数调用指令：

```
jal some_function
```

时，MIPS 会自动把"下一条指令的地址"保存到 `$ra` 中。这样函数结束时可以用：

```
jr $ra
```

跳回调用者那里继续执行。

所以 `$ra` 的作用类似于：

```
return 到哪里
```

---

2. 为什么要先保存 `$ra`?

图中的程序开头：

```
main:
    add $s7, $ra, $zero
```

等价于：

```
$s7 = $ra
```

也就是把 `$ra` 里的返回地址复制到 `$s7` 中。

结尾：

```
add $ra, $zero, $s7
jr $ra
```

等价于：

```
$ra = $s7
return;
```

这样做的目的通常是：

> *防止程序执行过程中 `$ra` 被别的指令覆盖，导致最后无法正确返回。*

---

3. 什么情况下 `$ra` 会被覆盖?

最典型的情况是程序中又调用了其他函数，例如：

```
main:
    jal func1
    jr $ra
```

执行 `jal func1` 时，`$ra` 会被改成 `jal func1` 后面那条指令的地址。

如果原来 `$ra` 中保存的是"返回主调过程"的地址，那么这个值就丢了。

因此，标准做法是：

```
main:
    addi $sp, $sp, -4
    sw $ra, 0($sp)
```

```
    jal func1

    lw $ra, 0($sp)
    addi $sp, $sp, 4
    jr $ra
```

也就是把 `$ra` 保存到栈里。

---

4. 但在图中这个程序里，有必要吗?

严格来说，**这个程序中没有必要保存** `$ra`。

因为图中的程序没有出现：

```
jal
```

也没有调用其他子程序，所以 `$ra` 在执行过程中不会被自动修改。

也就是说，这几句：

```
add $s7, $ra, $zero
...
add $ra, $zero, $s7
jr $ra
```

在当前程序里更像是一个教学写法，用来展示：

> *如果把这段程序看作一个"被调用的子程序"，那么应该保护并恢复返回地址。*

但对于这个具体例子来说，可以简化为：

```
main:
    la $s5, arr1
    la $s6, arr2
    addi $s0, $zero, 0x11

loop:
    addi $s0, $s0, -1
    sll $s1, $s0, 2
    add $s2, $s1, $s5
    lw $s3, 0($s2)
    add $s2, $s1, $s6
    sw $s3, 0($s2)
    bnez $s0, loop
```

如果是在模拟器中，还可以用系统调用退出程序，而不是 `jr $ra`。

---

5. 这里还有一个更规范的问题

图中用 `$s7` 保存 `$ra`：

```
add $s7, $ra, $zero
```

从功能上可以，但从调用约定上不算特别规范。

因为 `$s0` 到 `$s7` 是 **callee-saved registers**，也就是被调用函数如果要用它们，理论上也应该保存旧值并恢复。更规范的做法是把 `$ra` 存到栈里：

```
addi $sp, $sp, -4
sw $ra, 0($sp)

# 程序主体

lw $ra, 0($sp)
addi $sp, $sp, 4
jr $ra
```

---

6. 总结

图中这样写：

```
add $s7, $ra, $zero
...
add $ra, $zero, $s7
jr $ra
```

意思是：

> *先把返回地址 `$ra` 暂存到 `$s7`，程序结束前再恢复 `$ra`，最后通过 `jr $ra` 返回主调过程。*

但是在这段程序中，因为没有调用其他函数、没有执行 `jal`，所以 `$ra` 实际上不会被破坏。它是一个**保护返回地址的示范写法**，不是当前程序必须要有的操作。

下面是一段从文件读取数据并写入另一个文件的MIPS汇编代码，请先整体分析各个寄存器和段落标志的作用，再逐句或逐段分析代码的含义和作用：

```
.data
in_buff: .space 512
out_buff: .space 512
input_file: .asciiz "example.in"
output_file: .asciiz "example.out"
comma: .asciiz ", "
.text
la $a0, input_file #input_file 是一个字符串
li $a1, 0 #flag 0为读取 1为写入
li $a2, 0 #mode is ignored 设置为0就可以了
li $v0, 13 #13 为打开文件的 syscall 编号
syscall # 如果打开成功， 文件描述符返回到$v0中
 move $a0,$v0 # 将文件描述符载入到 $a0中
 la $a1, in_buff #in_buff 为数据暂存区
 li $a2, 512 #读取512个byte
 li $v0, 14 #14 为读取文件的 syscall 编号
 syscall
 li $v0 16 #16 为关闭文件的 syscall 编号
 syscall

#地址初始化，循环变量赋值为N
la $s0, in_buff
la $s1, out_buff
lw $s2, 0($s0)
li $t0, 0

#地址加4 循环变量减1
for: addi $s0, $s0, 4
addi $t0, $t0, 1
lw $t1, 0($s0)
sw $t1, 0($s1)
addi $s1, $s1, 4
#打印整数
move $a0, $t1
li $v0, 1
syscall
#打印逗号
la $a0, comma
li $v0, 4
syscall
bne $t0 $s2 for

la $a0, output_file #output_file 是一个字符串
li $a1, 1 #flag 0为读取 1为写入
li $a2, 0 #mode is ignored 设置为0就可以了
li $v0, 13 #13 为打开文件的 syscall 编号
syscall # 如果打开成功， 文件描述符返回到$v0中

move $a0,$v0 # 将文件描述符载入到 $a0中
la $a1, out_buff #in_buff 为数据暂存区
sll $s2 $s2 2
move $a2, $s2 #写入512个byte
li $v0, 15 #15 为写入文件的 syscall 编号
syscall
#此时$a0 中的文件描述符没变
#直接调用 syscall 16 关闭它
li $v0 16 #16 为关闭文件的 syscall 编号
syscall
```

下面这段程序的整体功能可以概括为：

> *从 `example.in` 文件中读取最多 512 字节到 `in_buff`；*
> *把 `in_buff` 中的一组 word 数据复制到 `out_buff`；*
> *复制过程中把每个整数打印到控制台，并打印逗号；*
> *最后把 `out_buff` 中的数据写入 `example.out` 文件。*

不过要注意：这段代码更像是 **MARS/SPIM 模拟器环境下的 MIPS 汇编**，里面的文件系统调用 `syscall 13/14/15/16` 是模拟器提供的，不是裸机 MIPS CPU 本身提供的功能。

一、数据段整体分析

```
.data
in_buff: .space 512
out_buff: .space 512
input_file: .asciiz "example.in"
output_file: .asciiz "example.out"
comma: .asciiz ", "
```

1. `.data`

表示下面进入**数据段**，用于定义变量、数组、字符串、缓冲区等。

---

2. `in_buff: .space 512`

```
in_buff: .space 512
```

定义一个名为 `in_buff` 的缓冲区，预留 512 字节空间。

它的作用是：

> *从输入文件 `example.in` 中读出的内容先存放到这里。*

可以类比 C 语言：

```
char in_buff[512];
```

但在后续代码中，它又被当成 word 数组使用，也就是每 4 字节当成一个整数。

---

3. `out_buff: .space 512`

```
out_buff: .space 512
```

定义一个输出缓冲区，也预留 512 字节空间。

它的作用是：

> *从 `in_buff` 中复制出来的数据先存到 `out_buff`，最后再写入输出文件 `example.out`。*

类似：

```
char out_buff[512];
```

---

4. `input_file: .asciiz "example.in"`

```
input_file: .asciiz "example.in"
```

定义一个以 `\0` 结尾的字符串：

```
"example.in\0"
```

它保存的是输入文件名。

后面执行：

```
la $a0, input_file
```

就是把这个字符串的首地址传给系统调用，用来打开文件。

---

5. `output_file: .asciiz "example.out"`

```
output_file: .asciiz "example.out"
```

定义输出文件名字符串：

```
"example.out\0"
```

后面会用它打开输出文件。

---

6. `comma: .asciiz ", "`

```
comma: .asciiz ", "
```

定义一个字符串：

```
", \0"
```

它用于在控制台打印逗号和空格。

注意，它只是打印到控制台，不会被写入 `out_buff`，也不会写入 `example.out`。

二、正文段整体分析

```
.text
```

`.text` 表示下面进入**正文段**，也就是程序指令段。CPU 实际执行的指令写在这里。

这段程序可以分成四个部分：

1. 打开输入文件，读取数据到 `in_buff`，关闭输入文件；
2. 初始化地址和循环变量；
3. 循环复制数组数据，并打印到控制台；

4. 打开输出文件，把 `out_buff` 写入文件，关闭输出文件。

三、主要寄存器作用总览

这段程序中寄存器的作用如下：

寄存器	作用
`$a0`	系统调用参数 1；经常用于传文件名地址、文件描述符、打印内容
`$a1`	系统调用参数 2；用于传缓冲区地址、文件打开模式等
`$a2`	系统调用参数 3；用于传读取/写入字节数
`$v0`	系统调用编号；系统调用返回值也放在 `$v0`
`$s0`	指向 `in_buff` 当前读取位置的地址指针
`$s1`	指向 `out_buff` 当前写入位置的地址指针
`$s2`	保存数组长度 N，也就是需要复制的 word 个数
`$t0`	循环计数器
`$t1`	暂存当前从输入数组中读出的数据
`$ra`	本代码未使用，但通常保存返回地址

其中 `$s0` 和 `$s1` 是两个非常关键的“地址指针”：

```
$s0 -> 当前正在读的 in_buff 地址
$s1 -> 当前正在写的 out_buff 地址
```

循环中每处理一个整数，它们都会加 4，因为一个整数是一个 word，即 4 字节。

四、第一段：打开输入文件

```
la $a0, input_file #input_file 是一个字符串
li $a1, 0 #flag 0为读取 1为写入
li $a2, 0 #mode is ignored 设置为0就可以了
li $v0, 13 #13 为打开文件的 syscall 编号
syscall # 如果打开成功 ， 文件描述符返回到$v0中
```

逐句解释：

```
la $a0, input_file
```

把字符串 `"example.in"` 的首地址放入 `$a0`。

也就是：

```
$a0 = &input_file
```

系统调用打开文件时，需要知道文件名在哪里，所以把文件名字符串地址传给 `$a0`。

```
li $a1, 0
```

设置打开模式。

在 MARS 中，`$a1 = 0` 通常表示只读模式，即 read。

所以这里是：

```
以读取模式打开文件
```

```
li $a2, 0
```

设置 mode 参数。

在 MARS 的文件打开系统调用中，这个参数通常可以忽略，设为 0 即可。

```
li $v0, 13
```

设置系统调用编号为 13。

在 MARS 中：

```
syscall 13 = open file
```

即打开文件。

```
syscall
```

执行系统调用。

如果打开成功，文件描述符会返回到 `$v0` 中。

文件描述符可以理解为：

> *操作系统或模拟器分配给该文件的编号。*
> *后续读、写、关闭文件时，不再直接用文件名，而是用这个编号。*

类似 C 语言中的：

```
fd = open("example.in", O_RDONLY);
```

执行完后：

```
$v0 = 输入文件的文件描述符
```

五、第二段：读取输入文件到 `in_buff`

```
move $a0,$v0 # 将文件描述符载入到 $a0中
la $a1, in_buff #in_buff 为数据暂存区
li $a2, 512 #读取512个byte
li $v0, 14 #14 为读取文件的 syscall 编号
syscall
```

逐句解释：

```
move $a0, $v0
```

把刚才打开文件返回的文件描述符复制到 `$a0`。

因为读文件系统调用需要的第一个参数是文件描述符。

即：

```
$a0 = fd
```

```
la $a1, in_buff
```

把输入缓冲区 `in_buff` 的首地址放入 `$a1`。

即：

```
$a1 = &in_buff
```

读出的内容会放到这个地址开始的内存区域中。

```
li $a2, 512
```

设置最多读取 512 字节。

即：

```
最多从文件中读取 512 bytes
```

```
li $v0, 14
```

设置系统调用编号为 14。

在 MARS 中：

```
syscall 14 = read file
```

```
syscall
```

执行读文件操作。

执行后，文件内容被读入 `in_buff`。

此外，`$v0` 中通常会返回实际读取到的字节数。

不过这段代码没有使用这个返回值。

六、第三段：关闭输入文件

```
li $v0 16 #16 为关闭文件的 syscall 编号
syscall
```

这里有一个语法问题，标准写法应为：

```
li $v0, 16
syscall
```

`syscall 16` 表示关闭文件。

关闭文件时，通常需要：

```
$a0 = 文件描述符
```

前面读文件时 `$a0` 已经被设置成输入文件描述符，而且后面没有修改 `$a0`，所以这里可以直接调用关闭文件。

但从规范性来说，最好明确写成：

```
move $a0, 输入文件描述符
li $v0, 16
syscall
```

这段代码依赖于 `$a0` 没有被改动。

七、第四段：初始化地址和循环变量

```
#地址初始化，循环变量赋值为N
la $s0, in_buff
la $s1, out_buff
lw $s2, 0($s0)
li $t0, 0
```

逐句解释：

```
la $s0, in_buff
```

把 `in_buff` 的首地址放入 `$s0`。

也就是说：

```
$s0 = &in_buff
```

之后 `$s0` 会作为输入数组的读指针。

```
la $s1, out_buff
```

把 `out_buff` 的首地址放入 `$s1`。

即：

```
$s1 = &out_buff
```

之后 `$s1` 会作为输出数组的写指针。

```
lw $s2, 0($s0)
```

从 `in_buff` 的前 4 个字节读取一个 word，放入 `$s2`。

即：

```
$s2 = Memory[$s0 + 0]
```

由于 `$s0 = &in_buff`，所以：

```
$s2 = in_buff[0]
```

这说明输入文件的数据格式被假定为：

```
第一个 word 表示数组长度 N
后面的 N 个 word 表示真正要复制的数据
```

例如，如果输入缓冲区内容是：

```
17, 1, 2, 3, ..., 17
```

那么：

```
$s2 = 17
```

表示后面有 17 个整数要复制。

```
li $t0, 0
```

初始化循环计数器：

```
$t0 = 0
```

之后 `$t0` 用来记录已经复制了多少个整数。

八、第五段：循环复制数据

核心循环如下：

```
for: addi $s0, $s0, 4
addi $t0, $t0, 1
lw $t1, 0($s0)
sw $t1, 0($s1)
addi $s1, $s1, 4
```

这里的逻辑是：

> *每次循环从 `in_buff` 中读取一个 word，写入 `out_buff`，然后地址向后移动 4 字节。*

1. `for:` 是标签

```
for:
```

这是循环起点标签。

后面的：

```
bne $t0, $s2, for
```

会根据条件跳回这里。

2. 输入指针前移 4 字节

```
addi $s0, $s0, 4
```

因为 `in_buff` 的第一个 word 是 N，不是要复制的数据，所以先让 `$s0` 加 4，跳过第一个 word。

原来：

```
$s0 = &in_buff[0]
```

执行一次后：

```
$s0 = &in_buff[1]
```

注意这里的 `in_buff[1]` 是按 word 理解的，不是按 byte 理解的。

因为每个 word 占 4 字节。

3. 循环计数器加 1

```
addi $t0, $t0, 1
```

表示已经准备复制第 `$t0` 个数据。

第一次循环后：

```
$t0 = 1
```

第二次循环后：

```
$t0 = 2
```

一直到：

```
$t0 = N
```

4. 从输入缓冲区读取当前 word

```
lw $t1, 0($s0)
```

从 `$s0` 当前指向的地址读取一个 word，放入 `$t1`。

第一次循环时：

```
$s0 = &in_buff + 4
```

所以读取的是：

```
$t1 = Memory[&in_buff + 4]
```

也就是第一个真正的数据。

5. 写入输出缓冲区

```
sw $t1, 0($s1)
```

把 `$t1` 中的数据写入 `$s1` 当前指向的位置。

第一次循环时：

```
$s1 = &out_buff
```

因此：

```
Memory[&out_buff] = 第一个数据
```

---

6. 输出缓冲区指针后移 4 字节

```
addi $s1, $s1, 4
```

因为刚刚已经写入了一个 word，下次应该写到下一个 word 的位置。

所以 `$s1` 后移 4 字节。

九、第六段：打印当前整数和逗号

```
#打印整数
move $a0, $t1
li $v0, 1
syscall
#打印逗号
la $a0, comma
li $v0, 4
syscall
```

这部分只是在控制台显示复制过程，不影响文件内容。

---

1. 打印整数

```
move $a0, $t1
```

把当前整数放入 `$a0`。

MARS 中打印整数的系统调用需要：

```
$a0 = 要打印的整数
$v0 = 1
```

```
li $v0, 1
syscall
```

`syscall 1` 表示打印整数。

所以这三句：

```
move $a0, $t1
li $v0, 1
syscall
```

作用是：

```
在控制台打印当前复制的整数
```

---

2. 打印逗号

```
la $a0, comma
```

把字符串 `", "` 的首地址放入 `$a0`。

```
li $v0, 4
syscall
```

`syscall 4` 表示打印字符串。

所以这三句：

```
la $a0, comma
li $v0, 4
syscall
```

作用是：

```
在控制台打印 ", "
```

十、第七段：判断循环是否结束

```
bne $t0 $s2 for
```

这里也有语法问题，标准写法应为：

```
bne $t0, $s2, for
```

意思是：

```
if ($t0 != $s2) goto for;
```

也就是：

> *如果已经复制的数据个数 `$t0` 还不等于总数 `$s2`，就继续循环。*

当：

```
$t0 = $s2
```

时，说明已经复制了 N 个整数，循环结束。

十一、循环过程举例

假设输入文件读入 `in_buff` 后，内存按 word 看是：

```
in_buff:
地址+0   17
地址+4   1
地址+8   2
地址+12  3
...
地址+68  17
```

其中第一个 word `17` 表示后面有 17 个数据。

初始化后：

```
$s0 = &in_buff
$s1 = &out_buff
$s2 = 17
$t0 = 0
```

第一次循环：

```
$s0 = $s0 + 4       # 指向第一个数据 1
$t0 = 1
$t1 = Memory[$s0]   # $t1 = 1
Memory[$s1] = 1     # out_buff[0] = 1
$s1 = $s1 + 4
```

第二次循环：

```
$s0 = $s0 + 4       # 指向第二个数据 2
$t0 = 2
$t1 = Memory[$s0]   # $t1 = 2
Memory[$s1] = 2     # out_buff[1] = 2
$s1 = $s1 + 4
```

直到第 17 次循环：

```
$t0 = 17
```

循环结束。

最终：

```
out_buff = 1, 2, 3, ..., 17
```

十二、第八段：打开输出文件

```
la $a0, output_file #output_file 是一个字符串
li $a1, 1 #flag 0为读取 1为写入
li $a2, 0 #mode is ignored 设置为0就可以了
li $v0, 13 #13 为打开文件的 syscall 编号
syscall # 如果打开成功 ，  文件描述符返回到$v0中
```

这一段和打开输入文件类似，只是文件名和打开模式不同。

```
la $a0, output_file
```

把 `"example.out"` 的地址放入 `$a0`。

```
li $a1, 1
```

设置打开模式为写入。

在 MARS 中，`$a1 = 1` 通常表示写入模式。

```
li $a2, 0
```

mode 忽略，设为 0。

```
li $v0, 13
syscall
```

打开输出文件。

打开成功后：

```
$v0 = 输出文件描述符
```

十三、第九段：写入输出文件

```
move $a0,$v0 # 将文件描述符载入到 $a0中
la $a1, out_buff #in_buff 为数据暂存区
sll $s2 $s2 2
move $a2, $s2 #写入512个byte
li $v0, 15 #15 为写入文件的 syscall 编号
syscall
```

逐句解释：

```
move $a0, $v0
```

把输出文件描述符放入 `$a0`。

写文件系统调用需要：

```
$a0 = 文件描述符
```

```
la $a1, out_buff
```

把输出缓冲区首地址放入 `$a1`。

写文件系统调用需要：

```
$a1 = 要写出的内存缓冲区地址
```

```
sll $s2 $s2 2
```

这里也有语法问题，标准写法应为：

```
sll $s2, $s2, 2
```

`sll` 是逻辑左移。

左移 2 位相当于乘以 4：

```
$s2 = $s2 * 4
```

因为 `$s2` 原来保存的是数据个数 N，而写文件时 `$a2` 需要的是**字节数**。

一个 word = 4 字节，所以：

```
写入字节数 = N * 4
```

例如：

```
$s2 = 17
```

执行：

```
sll $s2, $s2, 2
```

后：

```
$s2 = 68
```

表示需要写入 68 字节。

```
move $a2, $s2
```

把写入字节数放入 `$a2`。

写文件系统调用需要：

```
$a2 = 写入多少字节
```

注意原注释写的是：

```
#写入512个byte
```

这个注释不准确。

实际写入的是：

```
N * 4 bytes
```

不是 512 bytes。

```
li $v0, 15
```

设置系统调用编号为 15。

在 MARS 中：

```
syscall 15 = write file
```

```
syscall
```

执行写文件。

这会把 `out_buff` 中前 `N * 4` 字节写入 `example.out`。

注意：这里写入的是**二进制整数数据**，不是文本形式的 `"1, 2, 3, ..."`。

也就是说，控制台打印的是可读的整数和逗号，但输出文件中写入的是 raw binary word 数据。

十四、第十段：关闭输出文件

```
#此时$a0 中的文件描述符没变
#直接调用 syscall 16 关闭它
li $v0 16 #16 为关闭文件的 syscall 编号
syscall
```

同样，标准写法应为：

```
li $v0, 16
syscall
```

由于 `$a0` 仍然保存输出文件描述符，所以可以直接关闭输出文件。

十五、用类 C 语言理解整段程序

这段代码大致相当于下面的逻辑：

```c
char in_buff[512];
char out_buff[512];

int fd_in = open("example.in", READ);
read(fd_in, in_buff, 512);
close(fd_in);

int *in = (int *)in_buff;
int *out = (int *)out_buff;

int N = in[0];

for (int i = 0; i < N; i++) {
    out[i] = in[i + 1];
    printf("%d, ", out[i]);
}

int fd_out = open("example.out", WRITE);
write(fd_out, out_buff, N * 4);
close(fd_out);
```

这说明输入文件的数据格式应当类似：

```
第 1 个 word：N
后面 N 个 word：要复制的数据
```

例如二进制意义上：

```
17, 1, 2, 3, ..., 17
```

程序会把后面的 17 个整数复制到输出缓冲区，并写入输出文件。

十六、几个需要特别注意的问题

1. 这段代码假设输入文件是“二进制 word 数据”

因为程序使用：

```
lw $s2, 0($s0)
lw $t1, 0($s0)
```

这说明它把 `in_buff` 中每 4 字节当成一个整数。

如果 `example.in` 是普通文本文件，例如内容是：

```
17 1 2 3 4 5
```

那么这段代码不能正确按整数读取。

因为文本中的字符 `'1'`、`'7'`、空格等都是 ASCII 字节，不等于二进制整数 17。

如果要读取文本数字，需要自己写字符解析代码，把 ASCII 字符转成整数。

---

2. `out_buff` 写入的是二进制，不是文本

虽然循环里有：

```
li $v0, 1
syscall
```

和：

```
li $v0, 4
syscall
```

它们会在控制台打印：

```
1, 2, 3, ...
```

但是写入文件的部分是：

```
li $v0, 15
syscall
```

它写的是 `out_buff` 中的原始二进制数据。

所以 `example.out` 不是一个普通文本文件，而是一个二进制文件。

---

3. 有几处语法需要补逗号

原代码中有几处应改成标准格式：

```
li $v0 16
```

应改为：

```
li $v0, 16
```

```
bne $t0 $s2 for
```

应改为：

```
bne $t0, $s2, for
```

```
sll $s2 $s2 2
```

应改为：

```
sll $s2, $s2, 2
```

---

4. 没有检查文件是否成功打开

`syscall 13` 打开文件后，如果失败，`$v0` 可能返回负数。

这段程序没有检查：

```
bltz $v0, error
```

所以如果文件不存在或路径错误，后面的读写可能出问题。

---

5. 没有检查实际读取字节数

读文件后：

```
syscall
```

`$v0` 通常返回实际读取到的字节数。

但这段程序没有检查读到了多少字节，而是直接假设 `in_buff` 中有完整数据。

更稳妥的程序应该检查：

```
$v0 >= 4 * (N + 1)
```

否则可能读到的数据不足。

十七、核心思路总结

这段程序的关键思想是：

```
in_buff 的第一个 word 保存 N
后面 N 个 word 是要复制的数据
out_buff 用来保存复制结果
```

循环中通过地址加 4 来访问下一个整数：

```
addi $s0, $s0, 4   # 输入地址前进一个 word
addi $s1, $s1, 4   # 输出地址前进一个 word
```

因为：

```
1 word = 4 bytes
```

所以每次处理一个整数，地址都要加 4。
这就是在汇编中手动实现数组下标访问的方式。

问题1：MIPS汇编程序中“.globl”是什么意思？是不是可加可不加？
问题2：在利用MIPS程序打开文件时，如果其不存在，是否会直接自动创建？而在其写入文件时，是在先前文件中已有的数据后追加写还是抹掉原有数据重新写？

问题1：`.globl` 是什么意思？是不是可加可不加？

`.globl` 是 **global** 的缩写，意思是：

> *把某个标签声明为“全局符号”，使它可以被其他文件、链接器或模拟器入口机制识别。*

常见写法是：

```
.globl main
main:
    # program body
```

它表示：

```
main 这个标签是全局可见的。
```

---

1. `.globl main` 的作用

例如：

```
.text
.globl main

main:
    li $v0, 10
    syscall
```

这里的 `.globl main` 告诉汇编器：

> *`main` 是一个重要入口标签，外部也可以引用它。*

在一些 MIPS 环境中，程序启动时会寻找 `main` 标签。如果 `main` 没有声明为 `.globl`，有些环境可能找不到入口，或者在多文件链接时无法引用它。

---

2. `.globl` 本身不是 CPU 指令

`.globl` 和 `.data`、`.text`、`.word` 一样，都是**汇编指示符**，不是 CPU 真正执行的指令。

它不会生成类似 `add`、`lw`、`sw` 那样的机器指令。

---

3. 是不是可加可不加？

要分情况。

如果是**单文件、在 MARS/SPIM 中直接运行**，很多时候即使不写：

```
.globl main
```

程序也可能可以运行。因为模拟器可能默认从 `.text` 段第一条指令开始执行。

但是从规范角度，建议写：

```
.text
.globl main
main:
```

尤其是当你的程序中有明确的 `main:` 标签时，最好加上 `.globl main`。

---

4. 什么时候必须加？

在以下情况中，`.globl` 很重要：

情况一：多文件汇编

例如一个文件中定义函数：

```
.globl func
func:
    jr $ra
```

另一个文件中调用：

```
jal func
```

如果 `func` 不声明为 `.globl`，其他文件可能无法访问它。

情况二：程序入口需要被外部识别

例如：

```
.globl main
main:
    ...
```

表示 `main` 是程序入口之一。

---

5. 简单总结

`.globl main` 的意思是：

> *声明 `main` 是全局标签，使它可以作为程序入口或被其他文件引用。*

对于你现在写的单文件 MIPS 程序，很多情况下不加也能跑；但为了规范，建议加上：

```
.text
.globl main
main:
    ...
```

问题2：MIPS 打开文件时，不存在会不会自动创建？写入时是追加还是覆盖？

这个问题要看你使用的模拟器。你前面代码中使用的是 MARS 风格的文件系统调用：

```
li $v0, 13   # open file
syscall
```

MARS 中打开文件时主要看 `$a1` 的 flag。

一、打开文件时是否自动创建？

你前面的代码中：

```
la $a0, input_file
li $a1, 0
li $a2, 0
li $v0, 13
syscall
```

这里：

```
$a1 = 0
```

表示**只读打开**。

如果文件不存在，一般不会自动创建，而是打开失败，`$v0` 返回负数。

所以对于输入文件：

```
li $a1, 0
```

含义是：

```
只读打开；如果文件不存在，不创建，返回错误。
```

因此规范写法应该检查：

```
syscall
bltz $v0, open_error
```

二、写入文件时是否自动创建？

你前面的输出文件代码是：

```
la $a0, output_file
li $a1, 1
li $a2, 0
li $v0, 13
syscall
```

这里：

```
$a1 = 1
```

在 MARS 中通常表示：

```
以写入方式打开；如果文件不存在，则创建。
```

所以对于输出文件：

```
li $a1, 1
```

如果 `example.out` 不存在，通常会自动创建。

三、写入时是追加还是覆盖？

关键还是看打开文件时的 flag。

在 MARS 中常见规则是：

`$a1` flag	含义
`0`	只读打开
`1`	写入打开；不存在则创建；通常会从文件开头写，原内容会被覆盖/截断
`9`	追加写入；不存在则创建；写到文件末尾

所以你代码中的：

```
li $a1, 1
```

表示普通写入模式。

这种情况下，如果 `example.out` 已经存在，通常不是在原文件末尾追加，而是**重新写入**，原有内容会被覆盖，很多情况下等价于清空原文件后再写。

如果想追加到原文件末尾，应使用：

```
li $a1, 9
```

也就是 append 模式。

四、`syscall 15` 本身只负责“写到当前文件位置”

写文件时使用：

```
move $a0, $v0
la $a1, out_buff
move $a2, $s2
li $v0, 15
syscall
```

这里的 `syscall 15` 只是：

> *从 `$a1` 指向的缓冲区中取 `$a2` 个字节，写入 `$a0` 指定的文件描述符。*

它到底是从文件开头写，还是从文件末尾写，取决于之前 `open file` 时使用的 flag。

也就是说：

```
li $a1, 1
```

通常是覆盖写；

```
li $a1, 9
```

才是追加写。

五、针对你代码的结论

你的输入文件打开部分：

```
la $a0, input_file
li $a1, 0
li $v0, 13
syscall
```

含义是：

```
只读打开 example.in。
如果 example.in 不存在，不会自动创建，而是打开失败。
```

你的输出文件打开部分：

```
la $a0, output_file
li $a1, 1
li $v0, 13
syscall
```

含义是：

```
以写入方式打开 example.out。
如果 example.out 不存在，会创建。
如果 example.out 已存在，通常会覆盖原内容，而不是追加。
```

如果想追加写入，应改成：

```
li $a1, 9
```

六、建议你在程序中加入打开失败判断

例如：

```
li $v0, 13
syscall

bltz $v0, open_error
move $s0, $v0
```

其中：

```
bltz $v0, open_error
```

意思是：

```
如果 $v0 < 0，说明打开文件失败，跳转到错误处理。
```

这样程序会更稳妥。

在我完成第一个实验时，老师给定的输入文件a.in应该存放在什么路径下？应当与我的exp1_1.asm文件在同一目录下，还是应当与Mars4_5.jar（即MARS启动入口）放在同一目录下？

一般来说，在 MARS 中使用：

```
input_file: .asciiz "a.in"
```

这种**相对路径文件名**时，`a.in` 的查找位置通常不是由 `exp1_1.asm` 所在目录决定，而是由 **MARS 程序运行时的工作目录**决定。

对你现在最常见的情况，结论可以这样记：

> *如果你是双击 `Mars4_5.jar` 启动 MARS，那么 `a.in` 通常应当和 `Mars4_5.jar` 放在同一个目录下，而不是必须和 `exp1_1.asm` 放在同一个目录下。*

---

1. 为什么不是和 `.asm` 文件同目录？

你的汇编代码中写的是：

```
input_file: .asciiz "a.in"
```

这只是告诉 MARS：

> *打开当前工作目录下名为 `a.in` 的文件。*

它并没有写：

```
"C:/.../exp1_1/a.in"
```

也没有告诉 MARS 去 `.asm` 文件所在目录查找。

所以，即使你的汇编文件是：

```
D:/MIPS/exp1_1/exp1_1.asm
```

但如果 MARS 的工作目录是：

```
D:/Tools/MARS/
```

那么它会优先去找：

```
D:/Tools/MARS/a.in
```

而不是：

```
D:/MIPS/exp1_1/a.in
```

---

2. 推荐做法

做法一：把 `a.in` 放在 `Mars4_5.jar` 同目录下

例如：

```
D:/MARS/
    Mars4_5.jar
    a.in
    exp1_1.asm
```

或者：

```
D:/MARS/
    Mars4_5.jar
    a.in

D:/MIPS/exp1_1/
    exp1_1.asm
```

后一种情况下，只要你是从 `D:/MARS/` 启动的 MARS，也通常可以找到 `a.in`。

做法二：直接使用绝对路径，最稳妥

例如 Windows 下：

```
input_file: .asciiz "D:/MIPS/exp1_1/a.in"
```

注意建议用 `/`，不要写成：

```
"D:\MIPS\exp1_1\a.in"
```

因为反斜杠 `\` 在字符串中可能引起转义问题。

也可以写成双反斜杠：

```
input_file: .asciiz "D:\\MIPS\\exp1_1\\a.in"
```

但更推荐：

```
input_file: .asciiz "D:/MIPS/exp1_1/a.in"
```

---

3. 如果你从命令行启动 MARS

例如你在命令行中进入某个目录：

```
cd D:/MIPS/exp1_1
java -jar D:/MARS/Mars4_5.jar
```

那么此时当前工作目录可能是：

```
D:/MIPS/exp1_1
```

这时 `a.in` 放在 `exp1_1.asm` 同目录下就可能可以被找到。

所以更准确地说：

> *`a.in` 应放在 MARS 的当前工作目录下。*
> *双击启动 MARS 时，通常可以近似理解为放在 `Mars4_5.jar` 同目录下。*

---

4. 给你的实验建议

为了避免路径问题，实验时推荐这样组织：

```
exp1_1/
    Mars4_5.jar
    exp1_1.asm
    a.in
```

也就是把：

```
Mars4_5.jar
exp1_1.asm
a.in
```

三者暂时都放在同一个文件夹里。

这样最不容易出错。运行成功后，如果老师要求提交 `.asm`，只提交代码即可；`a.in` 按实验要求另附或说明。

如果我想查看a.in文件中存的具体是什么数，应当怎么打开？现在我用vscode打开，显示“此文件是二进制文件或使用了不受支持的文本编码，所以无法在文本编辑器中显示。”，强行打开后显示是乱码

这是正常现象：你的 `a.in` 很可能不是**文本文件**，而是**二进制文件**。

也就是说，它里面存的不是字符形式的：

```
17 1 2 3 4 5 ...
```

而是直接按 **4 字节整数 word** 存储的数据，例如：

```
0x00000011
0x00000001
0x00000002
...
```

所以 VS Code 按文本方式打开时会乱码。

---

1. 为什么会乱码?

文本文件中，数字 `17` 实际存的是字符：

```
'1' '7'
```

对应 ASCII 大致是：

```
0x31 0x37
```

但二进制整数文件中，数字 `17` 直接存成一个 32 位整数：

```
0x00000011
```

在小端序下，文件中的 4 个字节通常显示为：

```
11 00 00 00
```

这些字节不是正常可显示字符，所以文本编辑器会提示“二进制文件”或显示乱码。

---

2. 推荐方法一：用 VS Code 的 Hex Editor 插件查看

在 VS Code 中可以安装官方插件：

```
Hex Editor
```

安装后：

1. 右键点击 `a.in`
2. 选择 `Open With...`
3. 选择 `Hex Editor`

打开后你会看到类似：

```
11 00 00 00  01 00 00 00  02 00 00 00 ...
```

如果按 **每 4 个字节一组**解释，并且采用小端序：

```
11 00 00 00  -> 0x00000011 -> 17
01 00 00 00  -> 0x00000001 -> 1
02 00 00 00  -> 0x00000002 -> 2
```

所以它表示：

```
17, 1, 2, ...
```

---

3. 推荐方法二：用 Python 直接读出整数

你可以在 `a.in` 所在文件夹中新建一个 Python 文件，例如 `read_bin.py`：

```python
import struct

filename = "a.in"

with open(filename, "rb") as f:
    data = f.read()

nums = []

for i in range(0, len(data), 4):
    chunk = data[i:i+4]
    if len(chunk) < 4:
```

```
        break

    # "<i" 表示小端序 signed int
    num = struct.unpack("<i", chunk)[0]
    nums.append(num)

print(nums)
```

然后运行：

```
python read_bin.py
```

如果文件内容是按小端序 32 位整数存储的，你会看到类似：

```
[17, 1, 2, 3, 4, 5, ..., 17]
```

其中第一个数通常是 `N`，表示后面有多少个数据。

---

4. 如果读出来结果很奇怪怎么办?

如果你用：

```
struct.unpack("<i", chunk)
```

读出来结果很奇怪，例如出现特别大的数，那么可能文件是**大端序**存储的。

可以把代码改成：

```
num = struct.unpack(">i", chunk)[0]
```

其中：

```
<i  小端序 signed int
>i  大端序 signed int
<I  小端序 unsigned int
>I  大端序 unsigned int
```

不过在 MARS 里做这种实验时，常见情况是小端序，因此一般先试：

```
"<i"
```

---

5. 也可以用 MARS 自己查看

如果你的程序把文件读入：

```
in_buff: .space 512
```

然后执行完 read syscall 后，可以在 MARS 的数据段窗口中查看 `in_buff` 附近的内存。

比如查看：

```
0x10010000
0x10010004
0x10010008
...
```

如果第一个 word 是 `17`，后面依次是 `1, 2, 3, ...`，就说明读入正确。

这种方法适合验证程序是否正确读取了文件。

---

6. 对你这个实验的关键理解

你的程序中有：

```
lw $s2, 0($s0)
```

说明程序把 `a.in` 的前 4 字节当成一个整数 `N`。

然后循环中：

```
lw $t1, 0($s0)
```

又继续每 4 字节读取一个整数。

所以 `a.in` 的格式应该是：

```
第 1 个 word: 数据个数 N
第 2 个 word: 第 1 个数据
第 3 个 word: 第 2 个数据
...
第 N+1 个 word: 第 N 个数据
```

因此，不能用普通文本方式打开它。应当用 **十六进制查看器**，或者用 **Python 按 4 字节整数读取**。

汇编大作业说明文档-2026春(1).pdf



现在我正在完成这份大作业，正在完成实验1，利用vscode及hex editor插件打开a.in文件后，显示结果如图，根据大作业要求，这应该是存放了什么数据?

根据大作业说明文档，实验 1-1 的 `a.in` 是一个**二进制格式**输入文件，里面包含 **1 和 10 两个整数**。说明文档第 5 页明确写到：实验 1-1 要“从 `a.in` 读取两个整数”，并且测试样例 `a.in` “包含 1 和 10 两个整数”，输入输出文件均为二进制格式。`汇编大作业说明文档-2026春.pdf`

你现在在 Hex Editor 中看到的是：

```
01 00 00 00  0A 00 00 00
```

这 8 个字节应当按 **每 4 字节一个整数** 来理解：

```
01 00 00 00
0A 00 00 00
```

由于 MARS/MIPS 实验中通常按**小端序**存放整数，所以：

```
01 00 00 00  ->  0x00000001  ->  1
0A 00 00 00  ->  0x0000000A  ->  10
```

因此，这个 `a.in` 文件中存放的就是两个 32 位整数：

```
1, 10
```

它不是文本形式的：

```
1 10
```

而是二进制形式的：

```
整数 1 的 4 字节表示 + 整数 10 的 4 字节表示
```

所以 VS Code 普通文本方式打开会乱码；用 Hex Editor 看才是正确方式。
你的截图显示的内容是正确的，正好对应实验 1-1 的测试样例输入。

在实验1-3中，通过syscall的9号指令申请分配内存时，系统分配的内存地址是与自己的系统有关吗？为什么我的系统给我分配到的内存地址是0x0000000a，这应当不在正常的text或dynamic data中吧？

不是。实验 1-3 中 `syscall 9` 分配出来的地址**不应当与你自己的 Windows/Mac/Linux 系统真实内存地址有关**，它是在 **MARS 模拟器的虚拟 MIPS 内存空间**中分配的地址。

正常情况下，`syscall 9` 返回的地址应该在 **dynamic data / heap 区域**，常见类似：

```
0x10040000
0x10040010
0x10040020
```

而不应该是：

```
0x0000000a
```

`0x0000000a` 十进制就是 `10`，这几乎可以确定**不是系统分配的内存地址**，而是你把某个普通整数误当成了地址。

---

1. `syscall 9` 的正确用法

MARS 中 9 号系统调用类似 C 语言中的 `malloc/new`，用于在堆区申请空间。

调用格式是：

```
li $v0, 9        # syscall 9: 申请内存
move $a0, $t0    # $a0 = 要申请的字节数
syscall
```

调用后：

```
$v0 = 分配得到的内存首地址
```

所以关键是：

```
$a0 放申请的字节数
$v0 放系统调用编号 9
syscall 之后，$v0 才变成返回地址
```

例如要申请 `n` 个整数空间，每个整数 4 字节：

```
# 假设 $s0 = n

sll $a0, $s0, 2    # $a0 = n * 4，申请 n 个 int 的空间
li $v0, 9          # syscall 9
syscall

move $s1, $v0      # $s1 保存数组首地址
```

执行后，`$s1` 才是动态数组的首地址。

---

2. 为什么你会看到 `0x0000000a`？

`0x0000000a` 等于十进制 `10`。这很可能是以下几种情况之一。

情况一：你看到的是输入的 `n`，不是分配得到的地址

如果你输入数组长度：

```
10
```

那么读入后：

```
$v0 = 10
```

如果此时你还没有执行 `syscall 9`，或者执行后没有立刻保存 `$v0`，你看到的 `$v0` 就可能是 `0x0000000a`。

例如：

```
li $v0, 5
syscall          # 读入整数 n

# 此时 $v0 = n
# 如果你输入 10，那么 $v0 = 0x0000000a
```

这时的 `$v0` 不是地址，而是你输入的整数。

情况二：你没有把申请字节数放进 `$a0`

错误写法可能类似：

```
li $v0, 9
sll $v0, $v0, 2
syscall
```

这是错的。`syscall 9` 要求：

```
$a0 = 申请的字节数
$v0 = 9
```

不能把申请大小放在 `$v0` 里。

情况三：`syscall 9` 返回地址后，你又把 `$v0` 改掉了

这是很常见的错误。

例如：

```
li $v0, 9
syscall

li $v0, 5
syscall
```

第一段执行后，`$v0` 里本来是动态内存地址；但第二段又用 `$v0` 做了读整数系统调用，所以原地址被覆盖了。

因此，申请完内存后要**立刻保存返回地址**：

```
li $v0, 9
syscall

move $s1, $v0    # 立刻保存数组首地址
```

之后访问数组时用 `$s1`，不要再依赖 `$v0`。

---

3. 正确的实验 1-3 动态数组申请流程

实验 1-3 要输入数组长度 `n`，然后申请 `n` 个整数的空间。推荐流程是：

```
# 读入 n
li $v0, 5
syscall
move $s0, $v0        # $s0 = n

# 申请 n * 4 字节
sll $a0, $s0, 2      # $a0 = n * 4
li $v0, 9            # syscall 9: sbrk / allocate heap memory
syscall
move $s1, $v0        # $s1 = 数组 a 的首地址
```

此时：

```
$s0 = 数组长度 n
$s1 = 数组首地址
```

如果你输入 `n = 10`，那么：

```
$s0 = 10
$a0 = 40
$s1 = 0x1004xxxx   # 正常应是类似这样的地址
```

而不是：

```
$s1 = 0x0000000a
```

---

4. `0x0000000a` 为什么不正常？

MIPS 在 MARS 中常见内存布局大致是：

```
0x00400000 附近：text 段，放程序指令
0x10010000 附近：static data 段，放 .data 中定义的数据
0x10040000 附近：dynamic data / heap，放 syscall 9 申请的空间
0x7fffeffc 附近：stack，放栈
```

而：

```
0x0000000a
```

非常靠近 0 地址，不属于正常的数据段、堆区、栈区或正文段。

如果你用它作为数组地址，例如：

```
sw $t0, 0($s1)
```

且 `$s1 = 0x0000000a`，那么大概率会产生地址错误，尤其是它还不是 4 字节对齐地址：

```
0x0000000a = 10
10 不是 4 的倍数
```

执行 `lw` / `sw` 访问 word 时，地址通常要求是 4 的倍数。

---

5. 你应该怎么检查?

在 MARS 中重点看这几步执行后的寄存器:

```
li $v0, 5
syscall
move $s0, $v0
```

此时如果输入 `10`:

```
$s0 = 0x0000000a
```

这是正常的，因为 `$s0` 保存的是数组长度。

然后执行:

```
sll $a0, $s0, 2
li $v0, 9
syscall
move $s1, $v0
```

此时应该看到:

```
$a0 = 0x00000028    # 40 字节
$s1 = 0x1004xxxx    # 动态数组首地址
```

如果你看到:

```
$s1 = 0x0000000a
```

说明你没有正确保存 `syscall 9` 的返回值，或者保存地址前 `$v0` 已经被别的系统调用覆盖了。

---

6. 结论

`syscall 9` 分配出的内存地址**不是由你的真实电脑系统决定的**，而是由 MARS 模拟器的虚拟内存布局决定的。正常返回地址应当在动态数据区，通常类似 `0x10040000`。

你得到的:

```
0x0000000a
```

几乎肯定不是分配出的内存地址，而是十进制整数 `10`，很可能是你输入的数组长度 `n`，或者是 `$v0` 被后续 syscall 覆盖后的值。正确做法是：执行 `syscall 9` 后，立刻用一个 `$s` 寄存器保存 `$v0` 中的返回地址。

如果希望在vscode中解析查看dat二进制文件，之前所说的hex editor能不能用？

可以，**VS Code 的 Hex Editor 可以用来查看 `.dat` 二进制文件**。但要注意：Hex Editor 只能直接显示**原始字节**，不会自动帮你把它"解析成整数序列"。你需要按实验规定的数据格式自己分组理解。

大作业说明中，实战应用部分的输入文件 `test.dat` 明确是**二进制文件**，不是普通文本文件。文档第 7 页和第 10 页都说明输入文件命名为 `test.dat`，且为二进制文件。`汇编大作业说明文档-2026春.pdf`

---

1. Hex Editor 能看到什么?

例如你打开一个 `.dat` 后看到：

```
06 00 00 00  03 00 00 00  01 00 00 00  04 00 00 00
05 00 00 00  08 00 00 00  07 00 00 00
```

这不是乱码，而是二进制数据的十六进制表示。

如果按 **每 4 字节一个 int**，并且按**小端序**解释：

```
06 00 00 00  ->  6
03 00 00 00  ->  3
01 00 00 00  ->  1
04 00 00 00  ->  4
05 00 00 00  ->  5
08 00 00 00  ->  8
07 00 00 00  ->  7
```

那么它表示的整数序列就是：

```
6 3 1 4 5 8 7
```

这正好对应文档中实战应用 2.1 的样例输入：

```
6 3 1 4 5 8 7
```

---

2. 关键：每 4 个字节看成一个整数

在你的 MIPS 程序中，如果使用：

```
lw $t0, 0($s0)
```

那么它一次读取的是 **1 word = 4 bytes**。

所以看 `.dat` 文件时，应该按这样的方式理解：

```
第 0\~3 字节     第 1 个整数
第 4\~7 字节     第 2 个整数
第 8\~11 字节    第 3 个整数
……
```

也就是在 Hex Editor 中每 **4 个十六进制字节**组成一个数。

---

3. 小端序怎么读?

MARS 中通常按小端序存储整数。

所以：

```
01 00 00 00
```

表示：

```
0x00000001 = 1
```

而不是：

```
0x01000000
```

同理：

```
0A 00 00 00
```

表示：

```
0x0000000A = 10
```

如果看到：

```
E8 03 00 00
```

应解释为：

```
0x000003E8 = 1000
```

4. Hex Editor 适合“查看”，但不适合大量解析

如果文件很短，比如实验样例，Hex Editor 足够用。

但如果 `.dat` 文件中有很多整数，例如上千个数据，用 Hex Editor 手动看会很麻烦。更推荐写一个 Python 脚本解析：

```python
import struct

filename = "test.dat"

with open(filename, "rb") as f:
    data = f.read()

nums = []
for i in range(0, len(data), 4):
    chunk = data[i:i+4]
    if len(chunk) < 4:
        break
    nums.append(struct.unpack("<i", chunk)[0])

print(nums)
```

其中：

```
"<i"
```

表示：

```
小端序、有符号 32 位整数
```

如果数据都是非负整数，用：

```
"<I"
```

也可以。

---

5. 结论

可以用 VS Code 的 Hex Editor 打开 `.dat` 文件，但它显示的是字节，不是直接显示整数。你需要按：

```
每 4 字节一组
小端序
解释为 32 位整数
```

来读取。

对于小样例，用 Hex Editor 看就够了；对于较大的 `test.dat`，建议用 Python 脚本一次性解析成整数序列。

在MIPS中，系统调用syscall有哪些？请把所有编号的系统调用全部罗列出来

Thinking

搜索MARS MIPS系统调用服务表

hwlabnitc.github.io dpetersanderson.github.io github.com studocu.vn asm-editor.specy.app studocu.com scribd.com student.cs.uwaterloo.ca stackoverflow.com doc.ic.ac.uk marz.utk.edu youtube.com weinman.cs.grinnell.edu medium.com es.wikipedia.org en.wikipedia.org zh.wikipedia.org

搜索多个网站和资源

✓ 思考了 11s

严格说，`syscall` 不是 MIPS 指令集本身规定的一套"统一编号表"。你现在实验用的是 **MARS 模拟器**，所以应以 **MARS 的 syscall 编号**为准。MARS 的帮助文档说明：调用系统服务时，先把服务编号放入 `$v0`，把参数放入 `$a0/$a1/$a2/$a3` 或 `$f12`，再执行 `syscall`；返回值通常在 `$v0`、`$a0` 或 `$f0` 中。(dpetersanderson.github.io)

下面按 MARS 4.5 的系统调用表整理。

一、MARS 常用 syscall 总表

编号 `$v0`	功能	参数	返回值
1	打印整数	`$a0` = 要打印的整数	无
2	打印单精度浮点数	`$f12` = float	无
3	打印双精度浮点数	`$f12` = double	无
4	打印字符串	`$a0` = 以 `\0` 结尾的字符串首地址	无
5	读取整数	无	`$v0` = 读入的整数
6	读取单精度浮点数	无	`$f0` = 读入的 float
7	读取双精度浮点数	无	`$f0` = 读入的 double
8	读取字符串	`$a0` = 缓冲区地址；`$a1` = 最多读取字符数	缓冲区中得到字符串
9	申请堆内存 `sbrk`	`$a0` = 申请字节数	`$v0` = 分配到的内存首地址
10	退出程序	无	无
11	打印字符	`$a0` = 字符 ASCII 码，实际用低 8 位	无
12	读取字符	无	`$v0` = 读入字符 ASCII 码
13	打开文件	`$a0` = 文件名字符串地址；`$a1` = flags；`$a2` = mode	`$v0` = 文件描述符，失败为负数
14	从文件读取	`$a0` = 文件描述符；`$a1` = 输入缓冲区地址；`$a2` = 最多读取字节数	`$v0` = 实际读取字节数，EOF 为 0，错误为负数
15	写入文件	`$a0` = 文件描述符；`$a1` = 输出缓冲区地址；`$a2` = 写入字节数	`$v0` = 实际写入字节数，错误为负数
16	关闭文件	`$a0` = 文件描述符	无
17	带返回值退出程序 `exit2`	`$a0` = 退出码	无

其中 1 到 17 基本是 SPIM 兼容服务，但 MARS 文档特别说明，文件打开服务 13 的细节与 SPIM 有差异；30 及以上是 MARS 专有扩展。(dpetersanderson.github.io)

二、MARS 专有扩展 syscall

编号 `$v0`	功能	参数	返回值
30	获取系统时间	无	`$a0` = 低 32 位；`$a1` = 高 32 位
31	MIDI 输出，异步	`$a0` = pitch；`$a1` = duration；`$a2` = instrument；`$a3` = volume	无
32	sleep 暂停	`$a0` = 暂停毫秒数	无
33	MIDI 输出，同步	`$a0` = pitch；`$a1` = duration；`$a2` = instrument；`$a3` = volume	无
34	以十六进制打印整数	`$a0` = 要打印的整数	无
35	以二进制打印整数	`$a0` = 要打印的整数	无
36	以无符号十进制打印整数	`$a0` = 要打印的整数	无
37-39	未使用	-	-
40	设置随机数种子	`$a0` = 随机数生成器 id；`$a1` = seed	无
41	生成随机整数	`$a0` = 随机数生成器 id	`$a0` = 随机整数
42	生成指定范围随机整数	`$a0` = 随机数生成器 id；`$a1` = 上界	`$a0` = `[0, upper bound)` 内随机整数

编号 `$v0`	功能	参数	返回值
43	生成随机 float	`$a0` = 随机数生成器 id	`$f0` = `[0.0, 1.0)` 内随机 float
44	生成随机 double	`$a0` = 随机数生成器 id	`$f0` = `[0.0, 1.0)` 内随机 double
45-49	未使用	-	-
50	ConfirmDialog 确认对话框	`$a0` = 消息字符串地址	`$a0` = 0 Yes，1 No，2 Cancel
51	InputDialogInt 输入整数对话框	`$a0` = 消息字符串地址	`$a0` = 输入整数；`$a1` = 状态码
52	InputDialogFloat 输入 float 对话框	`$a0` = 消息字符串地址	`$f0` = 输入 float；`$a1` = 状态码
53	InputDialogDouble 输入 double 对话框	`$a0` = 消息字符串地址	`$f0` = 输入 double；`$a1` = 状态码
54	InputDialogString 输入字符串对话框	`$a0` = 消息字符串地址；`$a1` = 输入缓冲区地址；`$a2` = 最大字符数	`$a1` = 状态码
55	MessageDialog 消息框	`$a0` = 消息字符串地址；`$a1` = 消息类型	无
56	MessageDialogInt 显示整数消息框	`$a0` = 消息字符串地址；`$a1` = 要显示的整数	无
57	MessageDialogFloat 显示 float 消息框	`$a0` = 消息字符串地址；`$f12` = 要显示的 float	无
58	MessageDialogDouble 显示 double 消息框	`$a0` = 消息字符串地址；`$f12` = 要显示的 double	无
59	MessageDialogString 显示字符串消息框	`$a0` = 第一段消息字符串地址；`$a1` = 第二段字符串地址	无

三、几个实验中最常用的 syscall

你这次大作业中，最常用的其实只有这些：

编号	功能	典型用途
1	print integer	打印结果
4	print string	打印提示语、换行、逗号等
5	read integer	从控制台读入整数
9	sbrk	实验 1-3 中动态申请数组空间
10	exit	正常结束程序
13	open file	打开 `a.in`、`a.out`、`test.dat`
14	read file	从二进制文件读取数据
15	write file	向二进制文件写入数据
16	close file	关闭文件

四、文件相关 syscall 的 flag

对你当前实验最重要的是 `syscall 13` 的 `$a1` 参数。MARS 文档说明，文件打开服务 13 支持的 flag 是：`0` 表示只读，`1` 表示只写且创建文件，`9` 表示只写、创建文件并追加；`mode` 参数会被忽略；文件描述符从 3 开始分配，0、1、2 分别对应标准输入、标准输出、标准错误。(dpetersanderson.github.io)

所以：

```
# 只读打开
li $v0, 13
la $a0, filename
li $a1, 0
li $a2, 0
syscall
```

```
# 写入打开，不存在则创建；通常覆盖原文件
li $v0, 13
la $a0, filename
li $a1, 1
li $a2, 0
syscall
```

```
# 追加写入，不存在则创建；写到文件末尾
li $v0, 13
la $a0, filename
li $a1, 9
li $a2, 0
syscall
```

五、一个重要提醒

不要把这些编号理解成“真实 MIPS CPU 自带的标准系统调用编号”。`syscall` 这条指令本身是 MIPS 指令，但 `$v0=1/4/5/13/...` 这些服务编号是 **MARS/SPIM 等模拟器约定出来的实验环境接口**。在 Linux/MIPS 真实系统中，系统调用编号、参数传递和行为都不是这张表。

Sources

MIPS syscall functions available in MARS dpetersanderson.github.io

在实验2-1中，我希望给dp表设置一个非常大的初始值代表不可达状态，即INF，这个数设置多少比较合适?

在实验 2-1 里，`INF` 的作用通常是给 `dp` 表设置一个“足够大、表示当前状态不可达”的初值。因此它应该满足：

```
INF > 输入序列中所有可能出现的实际数值
```

同时还要避免在 MIPS 有符号比较中被当成负数。

---

1. 推荐设置

如果你的程序中只是拿 `INF` 做比较、赋值，不对它做加法、乘法等运算，那么可以设为：

```
INF = 0x7fffffff
```

也就是 32 位有符号整数的最大值：

```
0x7fffffff = 2147483647
```

在 MIPS 中这是最大的正数。只要输入数据不会等于或超过它，它就可以安全地作为“无穷大”。

---

2. 为什么不能随便设得更大?

MIPS 中寄存器是 32 位。如果按照**有符号整数**理解：

```
0x7fffffff =  2147483647
0x80000000 = -2147483648
0xffffffff = -1
```

所以不能把 `INF` 设置成：

```
0xffffffff
```

或者：

```
0x80000000
```

因为它们在有符号比较中不是“很大的正数”，反而会被当成负数。

例如：

```
slt $t0, $a, $b
```

如果使用的是有符号比较，那么 `0xffffffff` 会被认为是 `-1`，不是最大值。

---

3. 为什么之前常用 `0x3fffffff`?

`0x3fffffff` 是：

```
0x3fffffff = 1073741823
```

它也是一个很大的正数，并且离溢出边界更远。

如果程序里可能出现类似：

```
addi $t0, $t0, 1
```

或者：

```
add $t0, $t0, $t1
```

并且 `$t0` 可能是 `INF`，那么用 `0x7fffffff` 容易发生溢出。用 `0x3fffffff` 会更保险。

但对于实验 2-1 的 `dp` 表，如果 `INF` 只是表示“不可达状态”，一般只是做比较和覆盖，不参与加法，那么：

```
0x7fffffff
```

也是可以的。

---

4. 结合实验 2-1 的数据范围

大作业说明中只明确给出实验 2-1 的长度范围：

```
N <= 2000
```

但没有在题面中明确给出序列元素的数值范围。`汇编大作业说明文档-2026春.pdf`

所以严格来说，仅凭题面无法保证输入元素一定小于 `0x3fffffff` 或 `0x7fffffff`。不过在这类课程大作业里，测试数据通常是普通 32 位整数，不会故意取到 `2147483647` 这种边界值。

因此实践上推荐：

```
INF: .word 0x7fffffff
```

或者在代码中：

```
li $t0, 0x7fffffff
```

---

5. 更稳妥的选择

如果你希望稳妥且不接近溢出边界，可以继续使用：

```
li $t0, 0x3fffffff
```

这个值已经远大于普通测试数据，同时不会在一般操作中轻易溢出。

但是前提是：

```
输入序列中的最大值 < 0x3fffffff
```

如果你担心测试数据可能非常大，例如接近 `2^31 - 1`，那么应该改用：

```
li $t0, 0x7fffffff
```

---

6. 最严谨的方法

如果想完全避免 `INF` 与真实数据冲突，最好不要只靠一个大数表示不可达，而是额外设置一个状态标记，例如：

```
dp_value[i]      保存状态值
dp_valid[i]      0 表示不可达，1 表示可达
```

这样即使输入数据本身等于 `0x7fffffff`，也不会和 `INF` 混淆。

但这会让汇编程序复杂很多。对于本实验，一般没必要。

结论

实验 2-1 中，`INF` 建议这样选：

```
li $t0, 0x7fffffff
```

如果你确定输入数据不会超过 `0x3fffffff`，并且希望避免靠近溢出边界，也可以用：

```
li $t0, 0x3fffffff
```

不要设置成 `0xffffffff`、`0x80000000` 或更大的十六进制数，因为在 32 位有符号比较中它们会被解释成负数。

🚀 Powered by ChatGPT Exporter

对于实验2-2树型网络，请你帮我想几个全面的测试数据，尽可能考虑corner cases，给出十进制输入、理论输出和输入的十六进制形式（用于存进dat文件）

Thinking

- **确定问题和规则**
  我正在理清用户的问题。似乎是在问如何计算信号经过树形网络的传播。目标是找出所需的最小放大器数量，并考虑信号强度和衰减的规定。
- **确定问题和规则**
- **计算十六进制和字节转换**
- **计算十六进制与字节转换**

✓ 思考了 1m 16s

下面给你一组比较全面的实验 2-2 测试数据。所有十六进制都按 **32 位有符号整数、小端序** 给出，也就是一个十进制整数 `x` 在 `.dat` 中占 4 字节。

例如：

```
10 -> 0A 00 00 00
```

实验 2-2 的输入格式是：

```
n
第1个点的连接信息：k v1 w1 v2 w2 ...
第2个点的连接信息：k v1 w1 v2 w2 ...
...
第n个点的连接信息：k v1 w1 ...
初始信号强度
```

下面默认判断规则是：信号经过边后强度必须 **大于 0**，即当前强度 `power > edge_weight` 才能传过去。如果 `edge_weight >= 初始强度`，即使装放大器也无法通过该边。

测试 1：只有 server 一个结点

用途：最小规模，`n=1`，不需要放大器。

十进制输入

```
1
0
5
```

理论输出

```
0
```

十六进制输入

```
01 00 00 00 00 00 00 00 05 00 00 00
```

测试 2：题目样例

用途：验证基本读入、建图、树上遍历、放大器计数。

十进制输入

```
4
2 2 3 3 1
2 1 3 4 2
1 1 1
1 2 2
4
```

理论输出

```
1
```

解释：`1 -> 2` 后信号为 `1`，无法继续经过 `2 -> 4` 的衰减 `2`，因此需要在结点 `2` 装一个放大器。

十六进制输入

```
04 00 00 00 02 00 00 00 02 00 00 00 03 00 00 00
03 00 00 00 01 00 00 00 02 00 00 00 01 00 00 00
03 00 00 00 04 00 00 00 02 00 00 00 01 00 00 00
```

```
01 00 00 00 01 00 00 00 01 00 00 00 02 00 00 00
02 00 00 00 04 00 00 00
```

测试 3：星型树，不需要放大器

用途：所有终端都能从 server 直接收到信号。

十进制输入

```
5
4 2 1 3 2 4 3 5 4
1 1 1
1 1 2
1 1 3
1 1 4
5
```

理论输出

```
0
```

解释：初始强度为 `5`，从 server 到所有叶子边的衰减分别为 `1,2,3,4`，传到叶子后仍为正数。

十六进制输入

```
05 00 00 00 04 00 00 00 02 00 00 00 01 00 00 00
03 00 00 00 02 00 00 00 04 00 00 00 03 00 00 00
05 00 00 00 04 00 00 00 01 00 00 00 01 00 00 00
01 00 00 00 01 00 00 00 01 00 00 00 02 00 00 00
01 00 00 00 01 00 00 00 03 00 00 00 01 00 00 00
01 00 00 00 04 00 00 00 05 00 00 00
```

测试 4：长链，需要多个放大器

用途：测试连续路径上多次放大。

十进制输入

```
5
1 2 3
2 1 3 3 3
2 2 3 4 3
2 3 3 5 3
1 4 3
5
```

理论输出

```
3
```

解释：路径为 `1-2-3-4-5`，每条边衰减为 `3`，初始强度为 `5`。
`1 -> 2` 后剩余 `2`，无法过下一条边，所以在 `2` 装；同理还需要在 `3`、`4` 装，共 `3` 个。

十六进制输入

```
05 00 00 00 01 00 00 00 02 00 00 00 03 00 00 00
02 00 00 00 01 00 00 00 03 00 00 00 03 00 00 00
03 00 00 00 02 00 00 00 02 00 00 00 03 00 00 00
04 00 00 00 03 00 00 00 02 00 00 00 03 00 00 00
03 00 00 00 05 00 00 00 03 00 00 00 01 00 00 00
04 00 00 00 03 00 00 00 05 00 00 00
```

测试 5：一个放大器同时帮助多个子结点

用途：测试“在某个分支点装一个放大器，可以同时改善它向多个子节点转发的能力”。

十进制输入

```
6
2 2 4 3 1
3 1 4 4 3 5 4
2 1 1 6 2
1 2 3
1 2 4
1 3 2
5
```

理论输出

```
1
```

解释：`1 -> 2` 后信号剩余 `1`，结点 `2` 不能继续发给 `4` 或 `5`，在结点 `2` 装一个放大器后即可同时发给 `4` 和 `5`。

十六进制输入

```
06 00 00 00 02 00 00 00 02 00 00 00 04 00 00 00
03 00 00 00 01 00 00 00 03 00 00 00 01 00 00 00
04 00 00 00 04 00 00 00 03 00 00 00 05 00 00 00
04 00 00 00 02 00 00 00 01 00 00 00 01 00 00 00
06 00 00 00 02 00 00 00 01 00 00 00 02 00 00 00
03 00 00 00 01 00 00 00 02 00 00 00 04 00 00 00
01 00 00 00 03 00 00 00 02 00 00 00 05 00 00 00
```

测试 6：存在边衰减等于初始强度，无法传遍

用途：测试不可达情况。因为信号经过边后必须仍然大于 `0`，所以衰减等于初始强度时无法通过。

十进制输入

```
3
1 2 5
2 1 5 3 1
1 2 1
5
```

理论输出

```
0
```

解释：边 `1 -> 2` 衰减为 `5`，初始强度也是 `5`，经过后强度为 `0`，不能到达结点 `2`。不论如何安装放大器，都无法让信号跨过这条边。

十六进制输入

```
03 00 00 00 01 00 00 00 02 00 00 00 05 00 00 00
02 00 00 00 01 00 00 00 05 00 00 00 03 00 00 00
01 00 00 00 01 00 00 00 02 00 00 00 01 00 00 00
05 00 00 00
```

测试 7：复杂树，需要两个放大器

用途：测试多分支、多层路径，以及放大器计数是否最小。

十进制输入

```
8
2 2 6 3 2
3 1 6 4 5 5 3
2 1 2 7 7
2 2 5 6 6
2 2 3 8 6
1 4 6
1 3 7
1 5 6
10
```

理论输出

```
2
```

解释：

- `1 -> 2` 后剩余 `4`，不能继续过 `2 -> 4` 的衰减 `5`，所以需要在结点 `2` 装一个放大器；
- 装完后 `2 -> 4` 到达结点 `4` 时剩余 `5`，不能继续过 `4 -> 6` 的衰减 `6`，所以需要在结点 `4` 装一个放大器；
- 其他分支可达。

所以最少需要 `2` 个。

十六进制输入

```
08 00 00 00 02 00 00 00 02 00 00 00 06 00 00 00
03 00 00 00 02 00 00 00 03 00 00 00 01 00 00 00
06 00 00 00 04 00 00 00 05 00 00 00 05 00 00 00
03 00 00 00 02 00 00 00 01 00 00 00 02 00 00 00
07 00 00 00 07 00 00 00 02 00 00 00 02 00 00 00
05 00 00 00 06 00 00 00 06 00 00 00 02 00 00 00
02 00 00 00 03 00 00 00 08 00 00 00 06 00 00 00
01 00 00 00 04 00 00 00 06 00 00 00 01 00 00 00
```

```
03 00 00 00 07 00 00 00 01 00 00 00 05 00 00 00
06 00 00 00 0A 00 00 00
```

测试 8：剩余强度恰好等于边衰减

用途：测试边界条件 `remaining == edge_weight`。因为传过去后为 `0`，所以仍然不能通过，需要放大器。

十进制输入

```
4
1 2 3
2 1 3 3 3
2 2 3 4 1
1 3 1
6
```

理论输出

```
1
```

解释：`1 -> 2` 后剩余 `3`，但 `2 -> 3` 衰减也是 `3`，直接转发会变成 `0`，因此需要在结点 `2` 装放大器。装完后 `2 -> 3` 剩余 `3`，再到 `4` 可以通过。

十六进制输入

```
04 00 00 00 01 00 00 00 02 00 00 00 03 00 00 00
02 00 00 00 01 00 00 00 03 00 00 00 03 00 00 00
03 00 00 00 02 00 00 00 02 00 00 00 03 00 00 00
04 00 00 00 01 00 00 00 01 00 00 00 03 00 00 00
01 00 00 00 06 00 00 00
```

使用这些数据时的注意点

你每次测试时，只需要把其中一个测试的十六进制内容写入 `test.dat`，不要把多个测试拼在一起。因为题目输入格式是一组数据，不是多组数据。

另外，`0` 这个输出有两种可能含义：

```
不需要放大器 -> 输出 0
无法传遍整个网络 -> 也输出 0
```

所以测试 1、3、6 都输出 `0`，但含义不同：测试 1、3 是“无需放大器”，测试 6 是“无法完成传输”。